Abstraction

1. Create an abstract class Shape

```
public abstract class Shape {
```

2. The Shape class has two abstract methods: calculateArea() and calculatePerimeter. Both the methods have a return type of void

```
public abstract class Shape {
    public abstract void calculateArea();

    public abstract void calculatePerimeter();
}
```

3. Create a class Quadrilateral which extends the abstract class Shape.

```
public class Quadrilateral extends Shape {
    private double side1;
    private double side2;
    private double side3;
    private double side4;

    public Quadrilateral(double side1, double side2, double side3, double side4) {
        this.side1 = side1;
        this.side2 = side2;
        this.side3 = side3;
        this.side4 = side4;
    }

    @Override
    public void calculateArea() {
        System.out.println(x:"Area calculation for a general quadrilateral is not implemented");
    }

    @Override
    public void calculatePerimeter() {
        double perimeter = side1 + side2 + side3 + side4;
        System.out.println("Perimeter of the quadrilateral: " + perimeter);
    }
}
```

4. Implement all the abstract method of the parent class

```java
abstract class Shape {
    public abstract void calculateArea();
    public abstract void calculatePerimeter();
}

class Quadrilateral extends Shape {
    public void calculateArea() {
        System.out.println(x:"Calculating area of Quadrilateral...");
    }

    public void calculatePerimeter() {
        System.out.println(x:"Calculating perimeter of Quadrilateral...");
    }
}

public class workshop5 {
    Run | Debug
    public static void main(String[] args) {
        Quadrilateral quadrilateral = new Quadrilateral();
        quadrilateral.calculateArea();
        quadrilateral.calculatePerimeter();
    }
}
```

5. Create an abstract class named Vehicle which consist of two methods: wheel and door. Both the methods have void return type and no parameters. The method wheel has no implementation.

```java
abstract class Vehicle {
    public abstract void wheel();

    public abstract void door();
}

class Bus extends Vehicle {
    public void wheel() {
        System.out.println(x:"Bus has wheels.");
    }

    public void door() {
        System.out.println(x:"Bus has doors.");
    }
}

public class main {
    Run | Debug
    public static void main(String[] args) {
        Bus myBus = new Bus();

        myBus.wheel();
        myBus.door();
    }
```

6. Create a class name Bus and extend the Vehicle class.

```java
abstract class Vehicle {
    public abstract void wheel();

    public abstract void door();
}

class Bus extends Vehicle {
    public void wheel() {
        System.out.println(x:"Bus has wheels.");
    }

    public void door() {
        System.out.println(x:"Bus has doors.");
    }
}

public class main {
    Run | Debug
    public static void main(String[] args) {
        Bus myBus = new Bus();

        myBus.wheel();
        myBus.door();
    }
```

Interface

7. Create an interface Animal. The Animal interface has two methods eat() and walk()

```java
public interface Animal {
    void eat();
    void walk();
}
```

8. Create another interface Printable. The Printable interface has a method called display();

```java
public interface Printable {
    void display();
}
```

9. Create a class Cow that implements the Animal and Printable interfaces

```java
public class Cow implements Animal, Printable {

    @Override
    public void eat() {
        System.out.println("Cow is eating grass");
    }

    @Override
    public void walk() {
        System.out.println("Cow is walking slowly");
    }

    @Override
    public void display() {
        System.out.println("Cow is a domestic animal");
    }
}
```

10. Create an interface LivingBeing

```java
public interface LivingBeing {
```

11. Create an method void specialFeature()

```java
public interface LivingBeing {
    void specialFeature();
}
```

Classes

12. Create 2 classes Fish and Bird that implements LivingBeing

```java
public class workshop5

interface LivingBeing {
    void specialFeature();
}

public class Fish implements LivingBeing {
    @Override
    public void specialFeature() {
        System.out.println(x:"Fish lives in water.");
    }
}

public class Bird implements LivingBeing {
    @Override
    public void specialFeature() {
        System.out.println(x:"Birds fly in sky.");
    }
}
```

13. The specialFeature should display special features of the respective class animal.

```java
public class workshop5

interface LivingBeing {
    void specialFeature();
}

public class Fish implements LivingBeing {
    @Override
    public void specialFeature() {
        System.out.println(x:"Fish lives in water.");
    }
}

public class Bird implements LivingBeing {
    @Override
    public void specialFeature() {
        System.out.println(x:"Birds fly in sky.");
    }
}
```

Exception

14. In the following program, which exception will be generated

```
public class Demo{
        public static void main(String[] args) {
  System.out.println(10/0);
        }
}
```

Handle the exception above by using try-catch.

→ In the provided program, the exception ArithmeticException will be generated because you're trying to divide by zero, which is not allowed in mathematics.

```java
public class workshop5 {
    Run | Debug
    public static void main(String[] args) {
        try {
            System.out.println(10 / 0);
        } catch (ArithmeticException e) {
            System.out.println(x:"Exception caught: Division by zero is not allowed.");
        }
    }
}
```

15. In the following program, which exception will be generated

```
public class Demo{
        public static void main(String[] args) {
  int[] age = {10,20,25,24,28,27,30,31,32};
        System.out.println(age[9]);
        }
}
```

Handle the exception by using throws keyword.

→ In the provided program, the exception ArrayIndexOutOfBoundsException will be generated because you're trying to access an element at index 9 in an array that has only 9 elements. The valid indices for this array are from 0 to 8.

```java
public class workshop5 {
    Run | Debug
    public static void main(String[] args) throws ArrayIndexOutOfBoundsException {
        int[] age = {10, 20, 25, 24, 28, 27, 30, 31, 32};
        System.out.println(age[9]);
    }
}
```